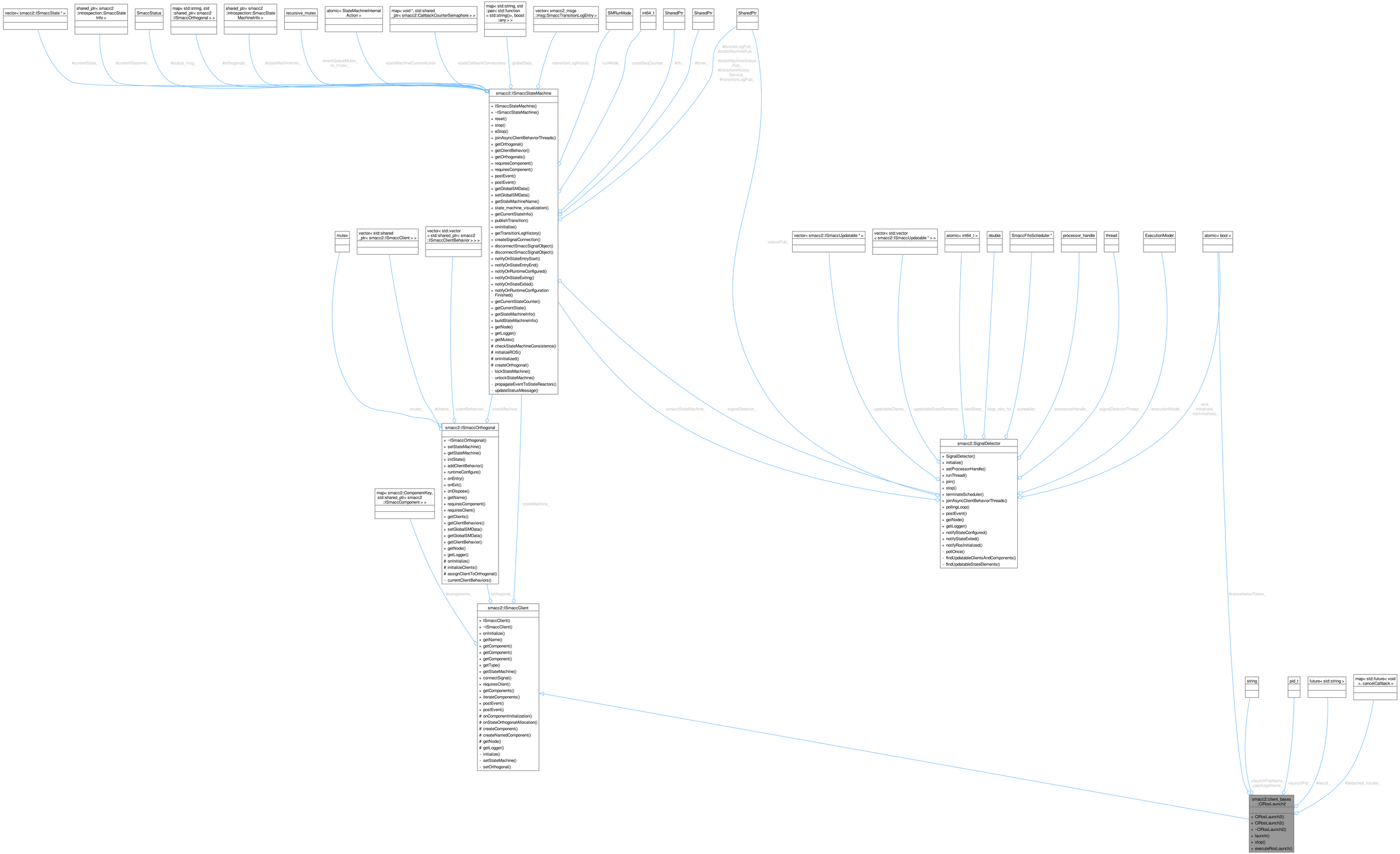
vector< smacc2::ISmaccState * >
shared_ptr< smacc2 ::introspection::SmaccState Info >
SmaccStatus
map< std::string, std ::shared_ptr< smacc2 ::ISmaccOrthogonal > >
shared_ptr< smacc2 ::introspection::SmaccState MachineInfo >
recursive_mutex
atomic< StateMachineInternal Action >
map< void *, std::shared _ptr< smacc2::CallbackCounterSemaphore > >
map< std::string, std ::pair< std::function < std::string()>, boost ::any > >
vector< smacc2_msgs ::msg::SmaccTransitionLogEntry >
SMRunMode
int64_t
SharedPtr
SharedPtr
SharedPtr
#currentState_
#currentStateInfo_
#status_msg_
#orthogonals_
#stateMachineInfo_
-eventQueueMutex_ -m_mutex_
-stateMachineCurrentAction
-stateCallbackConnections
-globalData_
-transitionLogHistory_
-runMode_
-stateSeqCounter_
#nh_
#timer_
#eventsLogPub_ #stateMachinePub
#stateMachineStatus Pub_
#transitionHistory Service_
#transitionLogPub_
smacc2::ISmaccStateMachine
+ ISmaccStateMachine()
+ ~ISmaccStateMachine()
+ reset()
+ stop()
+ eStop()
+ joinAsyncClientBehaviorThreads()
+ getOrthogonal()
+ getClientBehavior()
+ getOrthogonals()
+ requiresComponent()
+ requiresComponent()
+ postEvent()
+ postEvent()
+ getGlobalSMData()
+ setGlobalSMData()
+ getStateMachineName()
+ state_machine_visualization()
+ getCurrentStateInfo()
+ publishTransition()
+ onInitialize()
+ getTransitionLogHistory()
+ createSignalConnection()
+ disconnectSmaccSignalObject()
+ disconnectSmaccSignalObject()
+ notifyOnStateEntryStart()
+ notifyOnStateEntryEnd()
+ notifyOnRuntimeConfigured()
+ notifyOnStateExiting()
+ notifyOnStateExited()
+ notifyOnRuntimeConfiguration Finished()
+ getCurrentStateCounter()
+ getCurrentState()
+ getStateMachineInfo()
+ buildStateMachineInfo()
+ getNode()
+ getLogger()
+ getMutex()
# checkStateMachineConsistence()
# initializeROS()
# onInitialized()
# createOrthogonal()
- lockStateMachine()
- unlockStateMachine()
- propagateEventToStateReactors()
- updateStatusMessage()
mutex
vector< std::shared _ptr< smacc2::ISmaccClient > >
vector< std::vector < std::shared_ptr< smacc2 ::ISmaccClientBehavior > > >
-mutex_
#clients_
-clientBehaviors_
-stateMachine_
-smaccStateMachine_
-signalDetector_
smacc2::ISmaccOrthogonal
+ ~ISmaccOrthogonal()
+ setStateMachine()
+ getStateMachine()
+ initState()
+ addClientBehavior()
+ runtimeConfigure()
+ onEntry()
+ onExit()
+ onDispose()
+ getName()
+ requiresComponent()
+ requiresClient()
+ getClients()
+ getClientBehaviors()
+ setGlobalSMData()
+ getGlobalSMData()
+ getClientBehavior()
+ getNode()
+ getLogger()
# onInitialize()
# initializeClients()
# assignClientToOrthogonal()
- currentClientBehaviors()
map< smacc2::ComponentKey, std::shared_ptr< smacc2 ::ISmaccComponent > >
#components_
-orthogonal_
smacc2::ISmaccClient
+ ISmaccClient()
+ ~ISmaccClient()
+ onInitialize()
+ getName()
+ getComponent()
+ getComponent()
+ getComponent()
+ getType()
+ getStateMachine()
+ connectSignal()
+ requiresClient()
+ getComponents()
+ iterateComponents()
+ postEvent()
+ postEvent()
# onComponentInitialization()
# onStateOrthogonalAllocation()
# createComponent()
# createNamedComponent()
# getNode()
# getLogger()
- initialize()
- setStateMachine()
- setOrthogonal()
-statusPub_
vector< smacc2::ISmaccUpdatable * >
vector< std::vector < smacc2::ISmaccUpdatable * > >
atomic< int64_t >
double
SmaccFifoScheduler *
processor_handle
thread
ExecutionModel
atomic< bool >
-updatableClients_
-updatableStateElements_
-lastState_
-loop_rate_hz
-scheduler_
-processorHandle_
-signalDetectorThread_
-executionModel_
-end_ -initialized_ -rosInitialized_
smacc2::SignalDetector
+ SignalDetector()
+ initialize()
+ setProcessorHandle()
+ runThread()
+ join()
+ stop()
+ terminateScheduler()
+ joinAsyncClientBehaviorThreads()
+ pollingLoop()
+ postEvent()
+ getNode()
+ getLogger()
+ notifyStateConfigured()
+ notifyStateExited()
+ notifyRosInitialized()
- pollOnce()
- findUpdatableClientsAndComponents()
- findUpdatableStateElements()
#cancellationToken_
string
pid_t
future< std::string >
map< std::future< void >, cancelCallback >
+launchFileName_ +packageName_
-launchPid_
#result_
#detached_futures_
smacc2::client_bases ::ClRosLaunch2
+ ClRosLaunch2()
+ ClRosLaunch2()
+ ~ClRosLaunch2()
+ launch()
+ stop()
+ executeRosLaunch()